

<pre> #include<iostream.h> #include<conio.h> class first { public: int a,b; virtual void get()=0; //pure virtual funct. virtual void show()=0; //pure virtual funct. }; class second : public first { int c; public: void get(); void show(); }; void second::get() { cout<<"Enter Two NO="; cin>>a>>b; } </pre>	<pre> void second::show() { c=a+b; cout<<"\n Addition = "<<c; } void main() { first *x; second y; x=&y; x->get(); x->show(); getch(); } </pre>
--	--

In above example, *class 'first'* contains the pure virtual function therefore it is called as 'Abstract class' and object of abstract class cannot be created or instantiated.

Exception Handling in C++:

While doing programming in C++, there is possibility to happen errors. These errors may be encountered at two times-

- 1) Compile time error
- 2) Runtime error

Let's see these types of errors in details-

1) Compile time error:

- These type of errors detected by compiler at compile time of program.
- Such type of errors are due to wrong syntax of programming languages.
- Or when programmer violate the rules of writing C/C++ statements.
- These compiler error indicates something that must be fixed before the code can be compiled.
- All these errors are detected by compiler and thus are known as "compile-time errors"
- Most frequent compile time errors happen if we make following mistake:
 - Missing Parenthesis {}
 - Printing the value of variable without declaring it.
 - Missing semicolon at end of statement.
- Note that such type of errors are handled by compiler and it generate corresponding error message such that we can correct it.
- These type of errors are not harmful for computer because it is easily detected by compiler at compile time.

2) Run time error (Exception):

- An exception is a problem or error that arises during the execution of a program (program running).
- A C++ exception is a response to an exceptional situation that arises while a program is running, such as an attempt to divide a number by zero.
- Exceptions are run-time abnormal conditions that a program encounters during its execution.
- When an Exception occurs the normal flow of the program is disturbed and the program/Application/software terminates abnormally that cause's application failure which is harmful, which is not recommended, therefore, these exceptions should to be handled.

- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
 - A user has entered an invalid data.
 - A file that needs to be opened cannot be found.
 - A network connection has been lost in the middle of communications.

What happen if we do not handle exceptions?

- When an exception occurred, if we don't handle it, the program terminates abnormally and remaining program code will not get executed that cause's application failure which is harmful.

Example-

- We know that, an array is of fixed size and each element is accessed using the indices. Consider, we have created an array with size 7. Then the valid expressions to access the elements of this array will be a[0] to a[6] (length-1).
- Whenever, we used an -ve index value or, the index value greater than the size of the array, then there will be exception occurs.

How to handle exception in C++?

- It is necessary to handle exceptions because they may cause software or application failure.
- Exceptions provide a way to transfer control from one part of a program to another.
- In C++ exceptions are handled using three keywords: **try, catch, and throw.**
- Lets see these keywords in details-
- **try** –
- The try block/statement allows us to define a block of code that will be tested for errors while it is being executed.
- In short, A try block consists such line of code that may be a possibility of exceptions.
- **catch** –
- The catch block/statement allows us to define a block of code to be executed, if an error occurs in the try block.
- If any error occurs in try block then compiler transfers the control towards catch block.
- In short, catch block is executed if and only if any error happen in try block.
- The catch keyword indicates the catching of an exception.
- Note that- Single try block have multiple catch blocks each catch block has its own code.
- **throw** –
- The throw keyword throws an exception when a problem is detected in try block.
- This throw keyword used to create a custom error.
- A program throws an exception when a problem shows up. This is done using a throw keyword.
- The syntax for using try, catch and throw is as follows –

```
try
{
    // Block of code to try
    throw exception; // Throw an exception when a problem arise
}
catch ()
{
    // Block of code to handle errors
}
```

In above syntax-

- We use the try block to test some code. If that code leads some problem then exception occurs.
- Then, throw keyword throws an occurred exception.
- And catch block caught the thrown exception and take necessary action over exception.

Note:

- Single try block has multiple catch blocks. Here, compiler decides which catch block is executed depending upon thrown exception.
- Execution of catch block is depends on exception. If exception not occur in try block then catch block never executed.

Note:

- The exception handling programs (try, catch, throw keywords) are **not supported by TurboC DosBox.**
- To execute exception handling programs use **DEV C++ software.**

Following programs demonstrate exception handling in C++ using try, catch and throw keyword.

```
#include<iostream>
using namespace std;
int main()
{
    try
    {
        int age;
        cout<<"\nEnter Your Age= ";
        cin>>age;
        if (age >= 18)
        {
            cout << "You are Eligible for Voting.";
        }
        else
        {
            throw (age);
        }
    }
    catch (int myNum)
    {
        cout << "You must be at least 18 years old.\n";
        cout << "Your Age is: " << myNum;
    }
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
    try
    {
        float a,b,c;
        cout<<"\nEnter two no=";
        cin>>a>>b;
        c=a/b;
        if(b == 0)
        {
            throw(b);
        }
        else
        {
            cout<<"\nDivision="<<c;
        }
    }
    catch(float m)
    {
        cout<<"Number Not Divisible by "<<m;
    }
    return(0);
}
```

Following program shows exception handling with multiple catch blocks.

```
#include<iostream>
using namespace std;
int main()
{
    int a=2;

    try
    {
        if(a==1)
            throw a;           //throwing integer exception

        else if(a==2)
            throw 'A';         //throwing character exception

        else if(a==3)
            throw 4.5;         //throwing float exception

    }
    catch(int p)
    {
        cout<<"\nInteger Exception occur: "<<p;
    }
    catch(char q)
    {
        cout<<"\nCharacter Exception occur: "<<q;
    }
    catch(float r)
    {
        cout<<"\nInteger Exception occur:"<<r;
    }
    return(0);
}
```